

К(П)ФУ

Институт Математики и Механики

Кафедра аэрогидромеханики

Казанцев Р.В.

Казань

2024

Содержание

1 Установка программ **2**

1.1 Установка Python . . . 2

1.2 Установка Git . . . 3

1.3 Установка Visual Studio Code . . . 3

2 Выполнение домашнего задания №0 **3**

2.1 Обновление репозитория перед выполнением домашнего задания . . . 5

2.1.1 Первоначальная настройка . . . 5

2.1.2 Обновление перед выполнением нового домашнего задания . . . 6

3 Задание №1. Размытие изображения **6**

3.1 Часть 1. Добавление padding . . . 6

3.1.1 Задание . . . 7

4 Домашнее задание № 2. Решение нелинейных уравнений методом бисекции **9**

4.1 Постановка задачи . . . 9

4.2 Алгоритм . . . 9

5 Домашнее задание № 3. Двойной маятник: численное решение и неустойчивость **9**

5.1 Постановка задачи . . . 9

5.2 Вывод уравнений движения . . . 10

5.3 Параметры задачи . . . 11

5.4 Задание 1. Численное решение и анимация . . . 11

5.5 Задание 2. Проверка неустойчивости (чувствительность к начальным условиям) . . . 11

1 Установка программ

Перед выполнением домашнего задания необходимо установить Python, Git и редактор кода.

1.1 Установка Python

Скачайте Python с официального сайта:

https://www.python.org/downloads/windows/

1. Выберите **Download Windows installer (64-bit)**.
2. В окне установки обязательно поставьте галочку:

   ☐ **Add python.exe to PATH**.

3. После установки откройте терминал PowerShell и выполните команду:

   ```
   python --version
   ```

   Должна быть выведена установленная версия Python, например:

   ```
   Python 3.13.15
   ```

   Если команда не работает или отображается неправильная версия, попробуйте перезагрузить компьютер.

4. Проверьте установленную версию `pip`:

   ```
   python -m pip --version
   ```

   Например, результат может выглядеть следующим образом:

   ```
   pip 26.2.1 from
   ↪  C:\Users\docto\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip
   ↪  (python 3.13)
   ```

5. Обновите `pip`:

   ```
   python -m pip install --upgrade pip
   ```

6. Проверьте, откуда запускается Python:

   ```
   where.exe python
   ```

1.2 Установка Git

Скачайте Git с официального сайта:

https://git-scm.com/install/windows

После установки откройте PowerShell и выполните:

```
git --version
```

Например, результат может выглядеть следующим образом:

```
git version 2.55.0.windows.5
```

1.3 Установка Visual Studio Code

В качестве среды разработки рекомендуется установить Visual Studio Code:

https://code.visualstudio.com/docs/setup/windows

После установки откройте раздел **Extensions** и установите расширения:

- Python;
- Git.

После установки всех необходимых программ рекомендуется перезагрузить компьютер.

2 Выполнение домашнего задания №0

1. Делайте форк репозитория курса:

   https://github.com/kkazantsevrv/geom_course_2026

   Процесс создания форка был показан на практическом занятии.

2. Клонируйте свой репозиторий на компьютер.

   (a) Создайте папку для хранения учебных проектов. Например, `works`.

   (b) Откройте терминал в созданной папке.

   В Windows 11 это можно сделать, нажав правой кнопкой мыши внутри папки и выбрав соответствующий пункт меню для открытия терминала.

   (c) Выполните команду:

   ```
   git clone https://github.com/ВАШ_НИКНЕЙМ_GITHUB/geom_course_2026
   ```

   После выполнения команды будет создана папка курса:

   ```
   /works/geom_course_2026
   ```

   (d) Перейдите в папку репозитория:

```
cd geom_course_2026
```

(e) Создайте виртуальное окружение Python. В терминале выполните:

```
python -m venv .venv
```

Эта команда создаст папку `.venv`. По умолчанию папки, названия которых начинаются с точки, могут не отображаться в некоторых файловых менеджерах.

Рекомендуется самостоятельно разобраться, зачем необходимо виртуальное окружение Python и какие преимущества оно даёт при разработке программ.

(f) Активируйте виртуальное окружение:

```
..venv\Scripts\Activate.ps1
```

После этого для текущего проекта будет использоваться виртуальное окружение `.venv`. Все дополнительные библиотеки Python будут устанавливаться именно в это окружение.

(g) Установите необходимые библиотеки:

```
python -m pip install numpy pandas matplotlib
```

3. Создайте файл:

```
geom_course_2026/home_work_0/main.py
```

Программа должна выводить:

- версию Python;
- версию NumPy;
- версию Pandas;
- версию Matplotlib.

Запустить программу можно командой:

```
python.exe main.py
```

Это возможно благодаря тому, что ранее было создано и активировано виртуальное окружение Python.

Без виртуального окружения пришлось бы указывать полный путь к интерпретатору Python, например:

```
C:\Users\Ваш_Профиль\AppData\Local\Programs\Python\Python313\python.exe main.py
```

4. Далее необходимо сообщить Git, какие изменения произошли в файловой структуре проекта — какие файлы были добавлены или изменены. Для этого выполните:

```
git add .
```

5. Добавьте комментарий к выбранным изменениям и зафиксируйте их:

```
git commit -m "ура, 1 задание почти завершено"
```

6. Отправьте изменения на GitHub:

   ```
   git push
   ```

   При первом использовании команды может появиться окно с просьбой авторизоваться через браузер и подтвердить адрес электронной почты. В более сложном случае, git в терминале может попросить username и password от github, однако, начиная с 2024 года вместо пароля приходится вводить токен, сгенерированный в настройках аккаунта github.

   После успешной авторизации дальнейшая отправка изменений на GitHub будет выполняться автоматически при использовании команды `git push`.

2.1 Обновление репозитория перед выполнением домашнего задания

Исходный репозиторий курса

```
https://github.com/kkazantsevru/geom_course_2026
```

будет периодически обновляться. Поэтому перед началом выполнения нового домашнего задания (то есть с домашнего задания номер 1) необходимо обновить свой форк.

2.1.1 Первоначальная настройка

Эти действия необходимо выполнить только один раз. Перейдите в папку репозитория:

```
cd geom_course_2026
```

Посмотрите, какие удалённые репозитории уже подключены:

```
git remote -v
```

После создания форка и клонирования репозитория вы увидите примерно следующее:

```
origin  https://github.com/ВАШ_НИКНЕЙМ_GITHUB/geom_course_2026.git (fetch)
origin  https://github.com/ВАШ_НИКНЕЙМ_GITHUB/geom_course_2026.git (push)
```

Добавьте исходный репозиторий курса под именем `upstream`:

```
git remote add upstream https://github.com/kkazantsevru/geom_course_2026.git
```

Теперь проверьте список удалённых репозиториев:

```
git remote -v
```

Должны появиться два удалённых репозитория:

- `origin` — личный форк студента;
- `upstream` — исходный репозиторий курса.

2.1.2 Обновление перед выполнением нового домашнего задания

Перед началом каждого нового домашнего задания выполните:

```
git fetch upstream
```

Эта команда загружает информацию о новых изменениях из исходного репозитория курса. Обновите локальную копию:

```
git merge upstream/main
```

После этого отправьте обновления в свой форк на GitHub:

```
git push origin main
```

Таким образом, последовательность команд перед началом нового домашнего задания будет следующей:

```
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

После выполнения этих команд локальная копия репозитория и личный форк на GitHub будут содержать последние изменения из исходного репозитория курса.

3 Задание №1. Размытие изображения

В этом задании необходимо реализовать простейший фильтр размытия изображения с помощью библиотеки NumPy.

Изображение будем рассматривать как двумерный в NumPy. Чёрно-белое изображение имеет размер
`(N, M)`. Каждый элемент изображения содержит значение яркости соответствующего пикселя.

Например, для черно-белого

```
image[i, j] = 255
```

означает, что пиксель с индексами `(i, j)` имеет белый цвет.

3.1 Часть 1. Добавление padding

При обработке изображения часто необходимо обращаться к пикселям, находящимся около границы изображения.

Рассмотрим изображение

```
1 2 3
4 5 6
7 8 9
```

Если добавить `padding` шириной 2, получим:

```
0 0 | 0 0 0 | 0 0
0 0 | 0 0 0 | 0 0
----|-------|----
0 0 | 1 2 3 | 0 0
0 0 | 4 5 6 | 0 0
0 0 | 7 8 9 | 0 0
----|-------|----
0 0 | 0 0 0 | 0 0
0 0 | 0 0 0 | 0 0
```

То есть вокруг исходного изображения добавляется рамка заданной ширины, заполненная нулями. Возможен и другой вариант, например, padding-элементы заполняются крайними строками и столбцами:

```
1 1 | 1 2 3 | 3 3
1 1 | 1 2 3 | 3 3
----|-------|----
1 1 | 1 2 3 | 3 3
4 4 | 4 5 6 | 6 6
7 7 | 7 8 9 | 9 9
----|-------|----
7 7 | 7 8 9 | 9 9
7 7 | 7 8 9 | 9 9
```

3.1.1 Задание

1. Допишите функцию `pad_image` в файле

   `task1/blur.py`

   Сигнатура функции:

   ```
   def pad_image(image: np.ndarray, pad: int, mode: str) -> np.ndarray:
   ...
   ```

   Функция принимает 3 аргумента:

   - `image` — двумерный массив NumPy с изображением;
   - `pad` — ширина добавляемой рамки,
   - `mode` — способ реализации padding.

   Функция должна вернуть новое изображение, окружённое рамкой из нулей шириной `pad` если `mode = "zero"`, либо изображение окруженное повторяющимися строками и столбцами как в примере, если
   `mode = "edge"`. Исходное изображение должно находиться в центре результирующего массива.

2. Допишите функцию

```
def box_blur(image: np.ndarray, kernel_size: int, mode: str) -> np.ndarray:
...
```

Эта функция должна реализовать "метод скользящего окна"для размытия изображения. Очевидно, `kernel_size` должен быть нечетным числом, а
`pad = kernel_size // 2`.

3. Запустите функию `main()` командой `python.exe main.py`.

4 Домашнее задание № 2. Решение нелинейных уравнений методом бисекции

4.1 Постановка задачи

Требуется разработать программу для численного решения нелинейного уравнения вида:

$$f(x) = 0 \tag{4.1}$$

на заданном отрезке $[a, b]$ с использованием **метода бисекции** (метода деления отрезка пополам).

4.2 Алгоритм

1. Проверить выполнение условия наличия корня на концах отрезка: функция $f(x)$ должна быть непрерывной на отрезке $[a, b]$, а на его концах принимать значения разных знаков:

$$f(a) \cdot f(b) < 0. \tag{4.2}$$

2. На каждом шаге алгоритма отрезок $[a_k, b_k]$ делится пополам в точке $c_k = \frac{a_k + b_k}{2}$.

3. Вычисляется значение $f(c_k)$. Если $f(c_k) = 0$, то корень найден точно. В противном случае выбирается та половина отрезка $[a_k, c_k]$ или $[c_k, b_k]$, на концах которой функция имеет разные знаки.

4. Итерационный процесс продолжается до тех пор, пока длина текущего отрезка не станет меньше заданной точности ε ($b_k - a_k < \varepsilon$) либо пока абсолютная погрешность по значению функции не станет меньше ε.

5 Домашнее задание № 3. Двойной маятник: численное решение и неустойчивость

5.1 Постановка задачи

Рассматривается плоский двойной математический маятник (рис. 1). Невесомый нерастяжимый стержень длины L_1 закреплён шарнирно в точке O, на его конце находится точечный груз массы m_1. К грузу m_1 шарнирно прикреплён второй невесомый стержень длины L_2 с точечным грузом массы m_2 на конце. Система движется в вертикальной плоскости в однородном поле тяжести $\mathbf{g}$, трение отсутствует.

Положение системы задаётся двумя обобщёнными координатами: углами θ_1 и θ_2 отклонения стержней от вертикали (отсчитываются от направления вниз, против часовой стрелки). Угловые скорости: $\omega_1 = \dot{\theta}_1$, $\omega_2 = \dot{\theta}_2$.

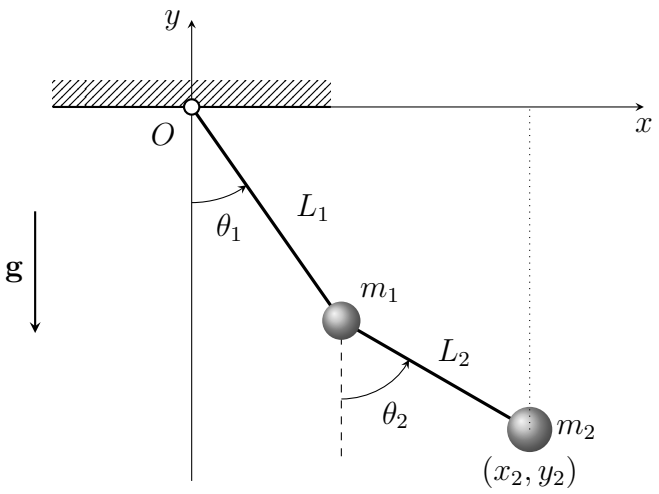


Рис. 1: Схема двойного маятника

5.2 Вывод уравнений движения

Декартовы координаты грузов:

$$\begin{aligned} x_1 &= L_1 \sin\theta_1, & y_1 &= -L_1\cos\theta_1, \\ x_2 &= x_1 + L_2\sin\theta_2, & y_2 &= y_1 - L_2\cos\theta_2. \end{aligned} \tag{5.1}$$

Кинетическая и потенциальная энергии системы:

$$T = \frac{m_1+m_2}{2}\,L_1^2\dot\theta_1^2 + \frac{m_2}{2}\,L_2^2\dot\theta_2^2 + m_2L_1L_2\,\dot\theta_1\dot\theta_2\cos(\theta_1-\theta_2), \tag{5.2}$$

$$V = -(m_1+m_2)\,gL_1\cos\theta_1 - m_2\,gL_2\cos\theta_2. \tag{5.3}$$

Функция Лагранжа $\mathcal{L} = T - V$. Уравнения Лагранжа второго рода

$$\frac{d}{dt}\frac{\partial\mathcal{L}}{\partial\dot\theta_i} - \frac{\partial\mathcal{L}}{\partial\theta_i} = 0, \qquad i = 1, 2,$$

дают (обозначим $\Delta = \theta_1 - \theta_2$):

$$\begin{aligned} (m_1+m_2)L_1\ddot\theta_1 + m_2L_2\ddot\theta_2\cos\Delta + m_2L_2\dot\theta_2^2\sin\Delta + (m_1+m_2)\,g\sin\theta_1 = 0, \\ L_2\ddot\theta_2 + L_1\ddot\theta_1\cos\Delta - L_1\dot\theta_1^2\sin\Delta + g\sin\theta_2 = 0. \end{aligned} \tag{5.4}$$

Система (5.4) линейна относительно $\ddot\theta_1$, $\ddot\theta_2$. Разрешая её, получаем **систему четырёх ОДУ первого порядка**, которую требуется решить численно:

$$\begin{aligned} \dot\theta_1 &= \omega_1, \\ \dot\omega_1 &= \frac{-g(2m_1+m_2)\sin\theta_1 - m_2g\sin(\theta_1-2\theta_2) - 2\sin\Delta\; m_2\,(\omega_2^2L_2 + \omega_1^2L_1\cos\Delta)}{L_1\,(2m_1+m_2-m_2\cos2\Delta)}, \\ \dot\theta_2 &= \omega_2, \\ \dot\omega_2 &= \frac{2\sin\Delta\,(\omega_1^2L_1(m_1+m_2) + g(m_1+m_2)\cos\theta_1 + \omega_2^2L_2m_2\cos\Delta)}{L_2\,(2m_1+m_2-m_2\cos2\Delta)}, \end{aligned} \tag{5.5}$$

где $\Delta = \theta_1 - \theta_2$. Вектор состояния: $\mathbf{y} = (\theta_1,\ \omega_1,\ \theta_2,\ \omega_2)^T$.

Полная механическая энергия $E = T + V$ сохраняется — это используется для контроля точности численного решения.

5.3 Параметры задачи

Параметр	Обозначение	Значение
Массы грузов	$m_1,\ m_2$	1 кг, 1 кг
Длины стержней	$L_1,\ L_2$	1 м, 1 м
Ускорение свободного падения	g	9.81 м/с^2
Интервал интегрирования	$[t_0, t_1]$	$[0,\ 20]$ с
Число точек вывода	N	1000
Начальные условия	$\mathbf{y}_0$	$(\pi/2,\ 0,\ \pi/2 + 0.01,\ 0)$
Точность интегратора	`rtol`, `atol`	10^{-8}

Таблица 1: Параметры двойного маятника

5.4 Задание 1. Численное решение и анимация

Написать программу на Python (`numpy`, `scipy`, `matplotlib`), которая:

1. Реализует функцию правых частей, возвращающую $\dot{\mathbf{y}}$ согласно (5.5).
2. Решает задачу Коши с параметрами из табл. 1 функцией `solve\_ivp` из модуля `scipy.integrate` методом `RK45`; результат выводить в N равноотстоящих точках (аргумент `t\_eval`).
3. Переводит решение в декартовы координаты по формулам (5.1).
4. Строит статичный график траектории нижнего груза $(x_2(t), y_2(t))$ с равным масштабом осей (`plt.axis('equal')`).
5. Строит анимацию движения маятника с помощью `FuncAnimation` из `matplotlib.animation`: на каждом кадре отображаются стержни $O \to m_1 \to m_2$ и след нижнего груза.

5.5 Задание 2. Проверка неустойчивости (чувствительность к начальным условиям)

Двойной маятник при больших начальных углах — хаотическая система: сколь угодно малое отличие начальных условий со временем приводит к полностью различным траекториям. Требуется проверить это численно.

1. Решить задачу дважды с почти одинаковыми начальными условиями:

$$\mathbf{y}_0^{(A)} = \left(\tfrac{\pi}{2},\ 0,\ \tfrac{\pi}{2} + 0.01,\ 0\right), \qquad \mathbf{y}_0^{(B)} = \left(\tfrac{\pi}{2},\ 0,\ \tfrac{\pi}{2} + 0.01 + \delta,\ 0\right), \qquad \delta = 10^{-6}.$$

2. Построить **две картинки рядом** — траектории нижнего груза для случаев A и B — в одном окне, используя `plt.subplots(1, 2)` с общими осями (`sharex=True`, `sharey=True`). В заголовке каждой панели указать значение $\theta_2(0)$.

3. Проверить, что расхождение траекторий — свойство системы, а не ошибка счёта: для случая A вычислить относительное изменение энергии $|E(t) - E(0)| \,/\, |E(0)|$ и построить его график. Повторить расчёт с `rtol = atol = 1e-5` и сравнить с результатом при 10^{-8}.